



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΥΠΟΛΟΓΙΣΤΙΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

ΛΕΙΤΟΥΡΓΙΚΑ ΣΥΣΤΗΜΑΤΑ ΥΠΟΛΟΓΙΣΤΩΝ
1^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ
ΚΛΗΣΕΙΣ ΣΥΣΤΗΜΑΤΟΣ, ΔΙΕΡΓΑΣΙΕΣ ΚΑΙ ΔΙΑΕΡΓΑΣΙΑΚΗ ΕΠΙΚΟΙΝΩΝΙΑ

ΑΓΓΕΛΟΣ ΚΑΜΑΡΙΑΔΗΣ | ΠΑΝΑΓΙΩΤΗΣ ΖΩΓΡΑΦΟΣ

ΑΝΑΓΝΩΣΗ ΚΑΙ ΕΓΓΡΑΦΗ ΑΡΧΕΙΩΝ ΣΤΗ C, ΜΕ ΤΗ ΒΟΗΘΕΙΑ ΚΛΗΣΕΩΝ ΣΥΣΤΗΜΑΤΟΣ

Ο ισοδύναμος κώδικας C, ο οποίος εκτυπώνει το πλήθος εμφανίσεων ενός χαρακτήρα σε κείμενο αποθηκευμένο στο αρχείο input.txt, στο αρχείο output.txt, φαίνεται παρακάτω.

```
angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 1...
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc != 4) {
        write(2, "Usage: program input output char\n", 33);
        exit(1);
    }

    int fd_in = open(argv[1], O_RDONLY);
    if (fd_in < 0) {
        perror("open input");
        exit(1);
    }

    int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd_out < 0) {
        perror("open output");
        exit(1);
    }

    char buffer[1024];
    ssize_t bytes;
    char target = argv[3][0];
    int count = 0;

    while ((bytes = read(fd_in, buffer, sizeof(buffer))) > 0) {
        for (int i = 0; i < bytes; i++) {
            if (buffer[i] == target) {
                count++;
            }
        }
    }

    if (bytes < 0) {
        perror("read");
        exit(1);
    }

    char result[50];
    int len = sprintf(result, "Count: %d\n", count);

    write(fd_out, result, len);

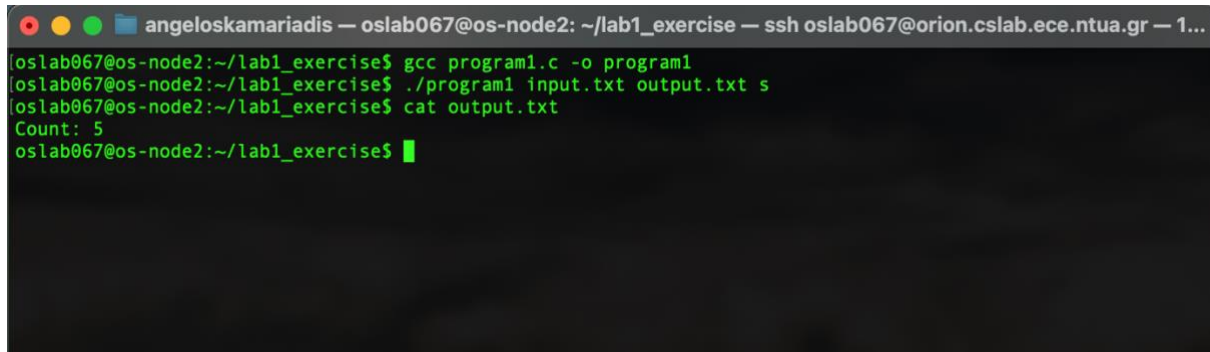
    close(fd_in);
    close(fd_out);

    return 0;
}
```

Το αρχείο input.txt είναι το εξής:

```
angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 1...
GNU nano 5.4                                input.txt
We are students at ece ntua, and this is our first lab
```

Κατά την εκτέλεση του κώδικα, πήραμε το παρακάτω αποτέλεσμα, το οποίο επαληθεύοντας με τη χρήση του αρχείου input.txt, παρατηρούμε πως είναι σωστό!



```
angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 1...
oslab067@os-node2:~/lab1_exercise$ gcc program1.c -o program1
oslab067@os-node2:~/lab1_exercise$ ./program1 input.txt output.txt s
oslab067@os-node2:~/lab1_exercise$ cat output.txt
Count: 5
oslab067@os-node2:~/lab1_exercise$
```

ΔΗΜΙΟΥΡΓΙΑ ΔΙΕΡΓΑΣΙΩΝ

Επεκτήναμε το παραπάνω πρόγραμμα, ούτως ώστε να δημιουργεί μία διεργασία παιδί, η οποία χαιρετά τον κόσμο, αναφέροντας το αναγνωριστικό της και το αναγνωριστικό του γονέα της. Ο γονέας εκτυπώνει το αναγνωριστικό του παιδιού και περιμένει τον τερματισμό του. Έπειτα, ορίσαμε μια μεταβλητή x , στον γονέα πριν τη δημιουργία του παιδιού στην οποία αναθέσαμε διαφορετική τιμή για τον γονέα και για το παιδί, ενώ ακολούθως εκτυπώσαμε την τιμή της.

Αναθέσαμε στη διεργασία παιδί την αναζήτηση του χαρακτήρα στο αρχείο, χωρίς όμως να της αναθέσουμε τον υπόλοιπο χειρισμό των αρχείων, και τέλος, δημιουργήσαμε μία διεργασία παιδί που εκτελεί τον κώδικα που σας δόθηκε στο Ερώτημα 1.

Παρατηρούμε πως όταν το παιδί αλλάζει την τιμή της x σε 20, δεν επηρεάζεται η τιμή της x στον γονέα (παραμένει 30). Αυτό αποδεικνύει ότι μετά τη `fork()`, η διεργασία παιδί αποκτά ένα ακριβές, αλλά ανεξάρτητο αντίγραφο της μνήμης (ξεχωριστό address space). Οι μεταβλητές δεν είναι κοινές. Αντίθετα, παρατηρούμε ότι οι file descriptors είναι κοινοί, για αυτό και το παιδί μπόρεσε να διαβάσει τα αρχεία που άνοιξε ο γονέας.

Όλα τα παραπάνω φαίνονται στους κώδικες που ακολουθούν.

angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 1...

```
int main(int argc, char *argv[]) {
    if (argc != 4) {
        write(2, "Usage: program input output char\n", 33);
        exit(1);
    }
    int fd_in = open(argv[1], O_RDONLY);
    if (fd_in < 0) {
        perror("open input");
        exit(1);
    }

    int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd_out < 0) {
        perror("open output");
        exit(1);
    }

    int x = 10;
    printf("[Parent] Variable x initialized to: %d\n", x);
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0) {
        printf("[Child] Hello Worlds! My PID is %d. My parent's PID is %d.\n", getpid(), getppid());
        x = 20;
        printf("[Child] I changed x to: %d\n", x);
        char buffer[1024];
        ssize_t bytes;
        char target = argv[3][0];
        int count = 0;
        while ((bytes = read(fd_in, buffer, sizeof(buffer))) > 0) {
            for (int i = 0; i < bytes; i++) {
                if (buffer[i] == target) {
                    count++;
                }
            }
        }
        if (bytes < 0) {
            perror("read");
            exit(1);
        }
        char result[50];
        int len = sprintf(result, "Count: %d\n", count);
        write(fd_out, result, len);
        exit(0);
    }
    else {
        printf("[Parent] I created the child whose PID is: %d\n", pid);
        x = 30;
        printf("[Parent] I changed x to: %d\n", x);
        wait(NULL);
        printf("[Parent] Child, terminated.\n");
        close(fd_in);
        close(fd_out);
    }
    return 0;
}
```

64,1 Bot

angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 1...

```
oslab067@os-node2:~/lab1_exercise$ ls
Makefile input.txt output.txt program1.c program2.c program3.c program4.c
a1.1-C.c output.rxt program1 program2 program3 program4
oslab067@os-node2:~/lab1_exercise$ vim program2.c
oslab067@os-node2:~/lab1_exercise$ gcc program2.c -o program
oslab067@os-node2:~/lab1_exercise$ ./program input.txt output.txt s
[Parent] Variable x initialized to: 10
[Parent] I created the child whose PID is: 3610733
[Parent] I changed x to: 30
[Child] Hello Worlds! My PID is 3610733. My parent's PID is 3610732.
[Child] I changed x to: 20
[Parent] Child, terminated.
oslab067@os-node2:~/lab1_exercise$
```

angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 1...

```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    if (argc != 4) {
        printf("Usage: %s input output char\n", argv[0]);
        exit(1);
    }
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0) {
        char *args[] = {"/part1", argv[1], argv[2], argv[3], NULL};
        execv("/part1", args);
        perror("execv failed");
        exit(1);
    }
    else {
        wait(NULL);
        printf("[Parent] The child (through syscall: execv) has completed the execution.\n");
    }
    return 0;
}
```

angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 117x57

```
oslab067@os-node2:~/lab1_exercise$ ls
Makefile askisi4 mai.c main.c      output.txt program1  program2  program2-2.c program3  worker
a1.1-C.c input.txt main  output.rxt program  program1.c program2-2 program2.c program3.c worker.c
oslab067@os-node2:~/lab1_exercise$ gcc program2.c -o program2
oslab067@os-node2:~/lab1_exercise$ gcc program2-2.c -o program2-2
oslab067@os-node2:~/lab1_exercise$ ./program2-2 input.txt output.txt s
[Parent] Variable x initialized to: 10
[Parent] I created the child whose PID is: 2079988
[Parent] I changed x to: 30
[Child] Hello Worlds! My PID is 2079988. My parent's PID is 2079987.
[Child] I changed x to: 20
[Parent] Child, terminated.
[Parent] The child (through syscall: execv) has completed the execution.
oslab067@os-node2:~/lab1_exercise$
```

ΔΙΑΔΙΕΡΓΑΣΙΑΚΗ ΕΠΙΚΟΙΝΩΝΙΑ

Επεκτείναμε το πρόγραμμα του παραπάνω ερωτήματος, ούτως ώστε να δημιουργεί P διεργασίες παιδιά, οι οποίες αναζητούν παράλληλα τον χαρακτήρα στο αρχείο, ενώ η γονεϊκή διεργασία συλλέγει και θα τυπώνει το συνολικό αποτέλεσμα. Όταν το πρόγραμμά δέχεται Control - C από το πληκτρολόγιο, αντί να τερματίζει, τυπώνει τον συνολικό αριθμό διεργασιών που αναζητούν το αρχείο.

Παρακάτω, φαίνεται η υλοποίηση μας.

```
angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.csilab.ece.ntua.gr — 126x59
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <signal.h>

#define P 3

volatile sig_atomic_t active_children = 0;

void safe_print_int(int num) {
    char buf[32];
    int i = 0;
    if (num == 0) {
        write(STDOUT_FILENO, "0", 1);
        return;
    }
    while (num > 0) {
        buf[i++] = (num % 10) + '0';
        num /= 10;
    }
    for (int j = i - 1; j >= 0; j--) {
        write(STDOUT_FILENO, &buf[j], 1);
    }
}

void sigint_handler(int sig) {
    write(STDOUT_FILENO, "\n[SIGNAL] Control+C caught! Active search processes: ", 53);
    safe_print_int(active_children);
    write(STDOUT_FILENO, "\n", 1);
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        write(STDERR_FILENO, "Usage: ./askisi3 input_file output_file char\n", 45);
        exit(1);
    }

    struct sigaction sa;
    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGINT, &sa, NULL);

    int pipefd[2];
    if (pipe(pipefd) == -1) {
        perror("pipe failed");
        exit(1);
    }

    int fd_temp = open(argv[1], O_RDONLY);
    if (fd_temp < 0) {
        perror("open input");
        exit(1);
    }
    off_t file_size = lseek(fd_temp, 0, SEEK_END);
    close(fd_temp);
```

1.1 Top


```

off_t chunk_size = file_size / P;

printf("[PARENT] Starting creation of %d children...\n", P);
printf("[PARENT] File size is %ld bytes. You have 5 seconds to press Ctrl+C!\n", (long)file_size);

for (int i = 0; i < P; i++) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0) {
        signal(SIGINT, SIG_IGN);
        close(pipefd[0]);

        sleep(5);

        int fd_in = open(argv[1], O_RDONLY);

        off_t start = i * chunk_size;
        off_t end = (i == P - 1) ? file_size : start + chunk_size;

        lseek(fd_in, start, SEEK_SET);

        char buffer[1024];
        ssize_t bytes;
        char target = argv[3][0];
        int local_count = 0;
        off_t bytes_to_read = end - start;
        off_t total_read = 0;

        while (total_read < bytes_to_read) {
            size_t to_read = sizeof(buffer);
            if (bytes_to_read - total_read < sizeof(buffer)) {
                to_read = bytes_to_read - total_read;
            }

            bytes = read(fd_in, buffer, to_read);
            if (bytes <= 0) break;

            for (int j = 0; j < bytes; j++) {
                if (buffer[j] == target) local_count++;
            }
            total_read += bytes;
        }
        close(fd_in);

        write(pipefd[1], &local_count, sizeof(int));
        close(pipefd[1]);
        exit(0);
    }
    else {
        active_children++;
    }
}

close(pipefd[1]);

```

117.1 67%

```

int total_count = 0;
int partial_count;

for (int i = 0; i < P; i++) {
    if (read(pipefd[0], &partial_count, sizeof(int)) > 0) {
        total_count += partial_count;
    }
}
close(pipefd[0]);

pid_t wpid;
int status;
while ((wpid = wait(&status)) > 0) {
    active_children--;
}

int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd_out >= 0) {
    char result_msg[50];
    int len = sprintf(result_msg, "Total Count: %d\n", total_count);
    write(fd_out, result_msg, len);
    close(fd_out);
}

printf("[PARENT] Search completed. Found %d occurrences.\n", total_count);

return 0;
}

```

146.1 Bot

```
angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 126x59
oslab067@os-node2:~/lab1_exercise$ vim program3.c
oslab067@os-node2:~/lab1_exercise$ gcc program3.c -o program3
oslab067@os-node2:~/lab1_exercise$ ./program3 input.txt output.txt s
[PARENT] Starting creation of 3 children...
[PARENT] File size is 55 bytes. You have 5 seconds to press Ctrl+C!
[PARENT] Search completed. Found 5 occurrences.
oslab067@os-node2:~/lab1_exercise$
```

ΕΦΑΡΜΟΓΗ ΠΑΡΑΛΛΗΛΗΣ ΚΑΤΑΜΕΤΡΗΣΗΣ ΧΑΡΑΚΤΗΡΩΝ

Στο ερώτημα 4, σχεδιάσαμε και υλοποιήσαμε μια προηγμένη, κατανεμημένη εφαρμογή παράλληλης επεξεργασίας σε C, βασισμένη στην αρχιτεκτονική Front - end/Dispatcher/Worker. Ουσιαστικά, χτίσαμε ένα έξυπνο σύστημα όπου ο χρήστης δίνει εντολές στο Front-end και ο Dispatcher αναλαμβάνει τον ρόλο του κεντρικού συντονιστή. Δηλαδή, τεμαχίζει ένα μεγάλο αρχείο σε μικρότερα κομμάτια και τα μοιράζει κυκλικά (round - robin) σε ένα δυναμικό πλήθος ανεξάρτητων εργατών (ένα ξεχωριστό πρόγραμμα που εκτελείται μέσω της `execv`). Αξιοποιώντας τις non - blocking σωληνώσεις για την αμφίδρομη ανταλλαγή μηνυμάτων, και σήματα για την παρακολούθηση της προόδου σε πραγματικό χρόνο, δημιουργήσαμε ένα πρόγραμμα που όχι μόνο λειτουργεί πραγματικά παράλληλα, αλλά διαθέτει και ανοχή σε σφάλματα, καθώς μπορεί να αντληφθεί τον αιφνίδιο "θάνατο" ενός εργάτη και να αναθέσει ξανά τη δουλειά του σε άλλον, διασφαλίζοντας την απόλυτη επιτυχία της αναζήτησης.

Τόσο ο κώδικας, όσο και η υλοποίηση του φαίνονται παρακάτω.

- worker.c

```
angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 126x59
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

int main(int argc, char *argv[]) {
    if (argc != 5) {
        fprintf(stderr, "[WORKER] Invalid arguments.\n");
        exit(1);
    }

    int worker_id = atoi(argv[1]);
    int fd_in = atoi(argv[2]);
    int fd_out = atoi(argv[3]);
    const char *filename = argv[4];

    int file_fd = open(filename, O_RDONLY);
    if (file_fd < 0) exit(1);

    char task_msg[64];
    while (read(fd_in, task_msg, 32) > 0) {
        long offset;
        size_t size;
        char target;

        sscanf(task_msg, "TSK %08ld %08ld %c", &offset, &size, &target);

        char buffer[1024];
        int local_count = 0;
        long total_read = 0;

        lseek(file_fd, offset, SEEK_SET);

        while (total_read < size) {
            long to_read = (size - total_read > sizeof(buffer)) ? sizeof(buffer) : (size - total_read);
            ssize_t bytes = read(file_fd, buffer, to_read);
            if (bytes <= 0) break;

            for (int i = 0; i < bytes; i++) {
                if (buffer[i] == target) local_count++;
            }
            total_read += bytes;
        }

        usleep(500000);

        char res_msg[16];
        sprintf(res_msg, "RES %05d %05d", worker_id, local_count);
        write(fd_out, res_msg, 16);
    }

    close(file_fd);
    close(fd_in);
    close(fd_out);
    return 0;
}
```

- main.c


```
angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 126x59

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <string.h>
#include <signal.h>
#include <sys/stat.h>
#include <sys/select.h>

#define MAX_WORKERS 20
#define MAX_CHUNKS 1000

typedef struct {
    int active;
    pid_t pid;
    int pipe_to_worker[2];
    int assigned_chunk;
} WorkerInfo;

typedef struct {
    long offset;
    long size;
    int status;
} ChunkInfo;

volatile sig_atomic_t status_requested = 0;

void sigusr1_handler(int sig) {
    status_requested = 1;
}

void set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}

void run_dispatcher(const char *filename, int pipe_fe_to_disp[2], int pipe_disp_to_fe[2], pid_t fe_pid) {
    close(pipe_fe_to_disp[1]);
    close(pipe_disp_to_fe[0]);

    set_nonblocking(pipe_fe_to_disp[0]);

    WorkerInfo workers[MAX_WORKERS];
    for (int i = 0; i < MAX_WORKERS; i++) workers[i].active = 0;

    ChunkInfo chunks[MAX_CHUNKS];
    int total_chunks = 0;
    int chunks_completed = 0;
    int total_found = 0;
    int search_in_progress = 0;
    char current_target = '\0';

    int pipe_work_to_disp[2];
    pipe(pipe_work_to_disp);
    set_nonblocking(pipe_work_to_disp[0]);

    struct sigaction sa;

    struct sigaction sa;
    sa.sa_handler = sigusr1_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    sigaction(SIGUSR1, &sa, NULL);

    char msg_buf[128];

    while (1) {
        int status;
        pid_t dead_pid = waitpid(-1, &status, WNOHANG);
        if (dead_pid > 0) {
            for (int i = 0; i < MAX_WORKERS; i++) {
                if (workers[i].active && workers[i].pid == dead_pid) {
                    workers[i].active = 0;
                    close(workers[i].pipe_to_worker[1]);

                    int c_id = workers[i].assigned_chunk;
                    if (c_id != -1 && chunks[c_id].status == 1) {
                        chunks[c_id].status = 0;
                    }
                    break;
                }
            }
        }

        if (status_requested) {
            status_requested = 0;
            char reply[128];
            if (!search_in_progress) {
                sprintf(reply, "STATUS: No search running. Found so far: %d\n", total_found);
            } else {
                int percent = (total_chunks == 0) ? 0 : (chunks_completed * 100) / total_chunks;
                sprintf(reply, "STATUS: Progress: %d%%. Chars found: %d\n", percent, total_found);
            }
            write(pipe_disp_to_fe[1], reply, strlen(reply) + 1);
            kill(fe_pid, SIGCONT);
        }

        if (read(pipe_fe_to_disp[0], msg_buf, sizeof(msg_buf)) > 0) {
            if (strncmp(msg_buf, "ADD", 3) == 0) {
                int added = 0;
                for (int i = 0; i < MAX_WORKERS; i++) {
                    if (!workers[i].active) {
                        pipe(workers[i].pipe_to_worker);
                        pid_t w_pid = fork();
                        if (w_pid == 0) {
                            close(pipe_fe_to_disp[0]); close(pipe_disp_to_fe[1]);
                            close(pipe_work_to_disp[0]); close(workers[i].pipe_to_worker[1]);

                            char id_str[10], in_str[10], out_str[10];
                            sprintf(id_str, "%d", i);
                            sprintf(in_str, "%d", workers[i].pipe_to_worker[0]);
                            sprintf(out_str, "%d", pipe_work_to_disp[1]);

                            char *args[] = {"/worker", id_str, in_str, out_str, (char *)filename, NULL};
                            execv("/worker", args);
                            exit(1);
                        }
                    }
                }
            }
        }
    }
}
```

```
angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 126x59

    } else {
        workers[i].active = 1;
        workers[i].pid = w_pid;
        workers[i].assigned_chunk = -1;
        close(workers[i].pipe_to_worker[0]);
        added = 1;
        break;
    }
}

char *resp = added ? "Worker added.\n" : "Max workers reached.\n";
write(pipe_disp_to_fe[1], resp, strlen(resp) + 1);
}

else if (strcmp(msg_buf, "SEARCH", 6) == 0) {
    current_target = msg_buf[7];
    struct stat st;
    stat(filename, &st);
    long file_size = st.st_size;
    long chunk_size = 1024;

    total_chunks = (file_size + chunk_size - 1) / chunk_size;
    for (int i = 0; i < total_chunks; i++) {
        chunks[i].offset = i * chunk_size;
        chunks[i].size = (i == total_chunks - 1) ? (file_size - i * chunk_size) : chunk_size;
        chunks[i].status = 0;
    }
    chunks_completed = 0;
    total_found = 0;
    search_in_progress = 1;

    char resp[] = "Search started in background.\n";
    write(pipe_disp_to_fe[1], resp, strlen(resp) + 1);
}

else if (strcmp(msg_buf, "EXIT", 4) == 0) {
    for (int i = 0; i < MAX_WORKERS; i++) if (workers[i].active) kill(workers[i].pid, SIGKILL);
    exit(0);
}

if (search_in_progress) {
    for (int c = 0; c < total_chunks; c++) {
        if (chunks[c].status == 0) {
            for (int w = 0; w < MAX_WORKERS; w++) {
                if (workers[w].active && workers[w].assigned_chunk == -1) {
                    workers[w].assigned_chunk = c;
                    chunks[c].status = 1;

                    char task_msg[32];
                    sprintf(task_msg, "TSK %08ld %08ld %c", chunks[c].offset, chunks[c].size, current_target);
                    write(workers[w].pipe_to_worker[1], task_msg, 32);
                    break;
                }
            }
        }
    }
}

char res_msg[16];

173.1 53%
```

```
angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 126x59

    if (read(pipe_work_to_disp[0], res_msg, 16) == 16) {
        int w_id, found;
        sscanf(res_msg, "RES %d %d", &w_id, &found);

        total_found += found;
        chunks[workers[w_id].assigned_chunk].status = 2;
        workers[w_id].assigned_chunk = -1;
        chunks_completed++;

        if (search_in_progress && chunks_completed == total_chunks) {
            search_in_progress = 0;
            char final_msg[128];
            sprintf(final_msg, "\n[DISPATCHER] Search complete! Total '%c' found: %d\n", current_target, total_found);
            write(pipe_disp_to_fe[1], final_msg, strlen(final_msg) + 1);
        }

        usleep(10000);
    }
}

void run_frontend(int pipe_fe_to_disp[2], int pipe_disp_to_fe[2], pid_t dispatcher_pid) {
    close(pipe_fe_to_disp[0]);
    close(pipe_disp_to_fe[1]);

    printf("=== ?~Za?~Daveµµévn ?~Qvaçñ?~Dñ?~Cñ Xa?~Aax?~Dñ?~A?~Iv (Front-End) ===\n");
    printf("?~Uv?~DoXé?~B: add, status, search <char>, exit\n");
    fflush(stdout);

    fd_set readfds;
    char input[128];
    char reply[256];

    while (1) {
        FD_ZERO(&readfds);
        FD_SET(STDIN_FILENO, &readfds);
        FD_SET(pipe_disp_to_fe[0], &readfds);

#ifdef MAX
#define MAX(a,b) ((a) > (b) ? (a) : (b))
#endif
        select(MAX(STDIN_FILENO, pipe_disp_to_fe[0]) + 1, &readfds, NULL, NULL, NULL);

        if (FD_ISSET(pipe_disp_to_fe[0], &readfds)) {
            if (read(pipe_disp_to_fe[0], reply, sizeof(reply)) > 0) {
                printf("%s", reply);
                if (strcmp(reply, "\n[DISP", 6) != 0) printf("> ");
                fflush(stdout);
            }
        }

        if (FD_ISSET(STDIN_FILENO, &readfds)) {
            if (fgetc(input, sizeof(input), stdin) == NULL) break;
            input[strcspn(input, "\n")] = 0;

            if (strcmp(input, "add") == 0 || strcmp(input, "search ", 7) == 0 || strcmp(input, "exit") == 0) {
                if (strcmp(input, "add") == 0) write(pipe_fe_to_disp[1], "ADD", 4);
                else if (strcmp(input, "exit") == 0) write(pipe_fe_to_disp[1], "EXIT", 5);
            }
        }
    }

231.1 81%
```

```

        else {
            char cmd[32];
            sprintf(cmd, "SEARCH %c", input[7]);
            write(pipe_fe_to_disp[1], cmd, strlen(cmd) + 1);
        }

        if (strcmp(input, "exit") == 0) break;
    }
    else if (strcmp(input, "status") == 0) {
        kill(dispatcher_pid, SIGUSR1);
    }
    else {
        printf("?~Fyv?~I?~C?~Dη ev?~Doλή.\n> ");
        fflush(stdout);
    }
}
}
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("X?~Aη?~Cη: ./askisi4 <α?~A?~GeLo_εl?~C?~Lδo?~E>\n");
        exit(1);
    }

    int pipe_fe_to_disp[2], pipe_disp_to_fe[2];
    pipe(pipe_fe_to_disp);
    pipe(pipe_disp_to_fe);

    pid_t dispatcher_pid = fork();
    if (dispatcher_pid == 0) {
        run_dispatcher(argv[1], pipe_fe_to_disp, pipe_disp_to_fe, getpid());
        exit(0);
    }

    run_frontend(pipe_fe_to_disp, pipe_disp_to_fe, dispatcher_pid);

    waitpid(dispatcher_pid, NULL, 0);
    return 0;
}

```

271.1

Bot

angeloskamariadis — oslab067@os-node2: ~/lab1_exercise — ssh oslab067@orion.cslab.ece.ntua.gr — 126x59

```

oslab067@os-node2:~/lab1_exercise$ ls
Makefile askisi4  main  output.rxt  program  program1.c  program2-2  program2.c  program3.c  worker.c
a1.1-C.c input.txt main.c output.txt program1  program2  program2-2.c  program3  worker
oslab067@os-node2:~/lab1_exercise$ vim worker.c
oslab067@os-node2:~/lab1_exercise$ gcc worker.c -o worker
oslab067@os-node2:~/lab1_exercise$ gcc main.c -o askisi4
oslab067@os-node2:~/lab1_exercise$ ./askisi4 input.txt
=== Καταμεμημένη Αναζήτηση Χαρακτήρων (Front-End) ===
Εντολές: add, status, search <char>, exit
> add
Worker added.
> search s
Search started in background.
>
[DISPATCHER] Search complete! Total 's' found: 5
> add
Worker added.

```

Εξέταση 1^{ου} Εργαστηρίου - Λειτουργικά Συστήματα

Υπολογιστών

Μέρος 1

- Ζητούμενο:

Υλοποίηση ενός προγράμματος το οποίο θα αναζητά πόσες φορές εμφανίζεται ένας χαρακτήρας μέσα σε ένα αρχείο εισόδου και θα γράφει το αποτέλεσμα σε ένα αρχείο εξόδου, μόνο με τη χρήση system-calls.

Τόσο τα αρχεία, όσο και ο χαρακτήρας προς αναζήτηση, θα δίνονται από τον χρήστη στη γραμμή εντολών.

Έτσι ώστε να μην δημιουργείται πρόβλημα αν ο χρήστης δεν δώσει τις παραμέτρους στη γραμμή εντολών σωστά, το πρόγραμμά θα πραγματοποιεί τους απαραίτητους ελέγχους και θα δίνει τις κατάλληλες οδηγίες χρήσης.

- Κώδικας

```
1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 #include <stdio.h>
5
6 int main(int argc, char *argv[]) {
7     if (argc != 4) {
8         write(STDOUT_FILENO, "Usage: program input output char\n", 33);
9         exit(1);
10    }
11    int fd_in = open(argv[1], O_RDONLY);
12    if (fd_in < 0) {
13        perror("open input");
14        exit(1);
15    }
16    int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
17    if (fd_out < 0) {
18        perror("open output");
19        exit(1);
20    }
21    char buffer[1024];
22    ssize_t bytes;
23    char target = argv[3][0];
24    int count = 0;
25    while ((bytes = read(fd_in, buffer, sizeof(buffer))) > 0) {
26        for (int i = 0; i < bytes; i++) {
27            if (buffer[i] == target) {
28                count++;
29            }
30        }
31    }
32    if (bytes < 0) {
33        perror("read");
34        exit(1);
35    }
36    char result[50];
37    int len = sprintf(result, "Count: %d\n", count);
38    write(fd_out, result, len);
39    close(fd_in);
40    close(fd_out);
41    return 0;
42 }
```

- Επεξήγηση κώδικα

Το πρόγραμμα διαβάζει ένα αρχείο εισόδου, μετράει πόσες φορές εμφανίζεται ένας συγκεκριμένος χαρακτήρας και γράφει το αποτέλεσμα σε ένα αρχείο εξόδου, χρησιμοποιώντας αποκλειστικά system calls. Δηλαδή, χαμηλού επιπέδου κλήσεις συστήματος αντί για τις κλασσικές εντολές τις C, "fopen" και "fclose"

- Πετυχαίνει τον στόχο του;

ΝΑΙ!

- Επεξήγηση τμημάτων του κώδικα

Έλεγχος Ορισμάτων

```
if (argc != 4) {  
    write(2, "Usage: program input output char\n", 33);  
    exit(1);  
}
```

Το πρόγραμμα απαιτεί ακριβώς 3 ορίσματα:

- `argv[1]` → αρχείο εισόδου
- `argv[2]` → αρχείο εξόδου
- `argv[3]` → ο χαρακτήρας προς αναζήτηση

Το `argc` μετράει και το ίδιο το όνομα του προγράμματος (`argv[0]`), οπότε ελέγχεται για 4. Αν δεν ισχύει, γράφει μήνυμα σφάλματος στο `stderr` και τερματίζει.

Άνοιγμα Αρχείων

```
int fd_in = open(argv[1], O_RDONLY);
```

Ανοίγει το αρχείο εισόδου σε κατάσταση μόνο ανάγνωση. Επιστρέφει έναν ακέραιο, τον file descriptor (`fd_in`). Αν αποτύχει, επιστρέφει -1.

```
int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

Ανοίγει/δημιουργεί το αρχείο εξόδου με τρεις flags

`O_WRONLY`: Μόνο εγγραφή

`O_CREAT`: Δημιουργία αν δεν υπάρχει

`O_TRUNC`: Αδειάζει το αρχείο αν ήδη υπάρχει

0644: Τα δικαιώματα 0644 σημαίνουν: ο ιδιοκτήτης έχει `read+write`, οι υπόλοιποι μόνο `read`.

Κύριος Βρόχος Ανάγνωσης

```
char buffer[1024];
```

```
ssize_t bytes;
```

```
char target = argv[3][0];
```

```
int count = 0;
```

```
while ((bytes = read(fd_in, buffer, sizeof(buffer))) > 0) {  
    for (int i = 0; i < bytes; i++) {  
        if (buffer[i] == target) {  
            count++;  
        }  
    }  
}
```

- Ο `target` είναι ο πρώτος χαρακτήρας του `argv[3]`
- Η `read()` διαβάζει έως 1024 bytes κάθε φορά και επιστρέφει πόσα bytes διάβασε πραγματικά

- Ο εσωτερικός βρόχος εξετάζει κάθε byte του buffer και αυξάνει τον count αν ταιριάζει
- Ο βρόχος σταματά όταν η read() επιστρέψει 0 (τέλος αρχείου) ή αρνητικό (σφάλμα)

Με αυτή τη λογική διαχειρίζεται σωστά και αρχεία μεγαλύτερα των 1024 bytes, αφού διαβάζει σε chunks.

Εγγραφή Αποτελέσματος

```
char result[50];
int len = sprintf(result, "Count: %d\n", count);
write(fd_out, result, len);
```

Μορφοποιεί το αποτέλεσμα σε string (π.χ. "Count: 42\n") και το γράφει στο αρχείο εξόδου.

Παράδειγμα Εκτέλεσης

```
./program input.txt output.txt 'a'
```

Μετράει πόσες φορές εμφανίζεται το 'a' στο input.txt και γράφει το αποτέλεσμα στο output.txt.

- System Calls που χρησιμοποιεί ο κώδικας
 - open()
 - read()
 - write()
 - close()
 - exit()

Γιατί χρειάζονται οι 4 βιβλιοθήκες;

Κάθε header file δίνει στο πρόγραμμά πρόσβαση σε συγκεκριμένες συναρτήσεις ή σταθερές που χρησιμοποιούνται παρακάτω στον κώδικα:

1. **<fcntl.h> (File Control):** Περιέχει τα flags και τις σταθερές που χρειάζονται για το άνοιγμα των αρχείων. Στον κώδικά, παρέχει τις σταθερές O_RDONLY, O_WRONLY, O_CREAT και O_TRUNC, οι οποίες περνιούνται ως ορίσματα στη συνάρτηση open().
2. **<unistd.h> (Unix Standard):** Παρέχει πρόσβαση στο API του λειτουργικού συστήματος (POSIX) για βασικές λειτουργίες I/O χαμηλού επιπέδου. Παρέχει τις βασικές συναρτήσεις διαχείρισης αρχείων: read(), write() και close(). Επίσης, ορίζει τον τύπο ssize_t.
3. **<stdlib.h> (Standard Library):** Είναι η γενική βιβλιοθήκη της C για βασικές λειτουργίες όπως η διαχείριση μνήμης, μετατροπές τύπων και ο έλεγχος του προγράμματος. Στον κώδικά, χρησιμοποιείται αποκλειστικά για τη συνάρτηση exit(1), η οποία τερματίζει απότομα το πρόγραμμα

επιστρέφοντας έναν κωδικό σφάλματος στο λειτουργικό σύστημα αν κάτι πάει στραβά.

4. **<stdio.h> (Standard Input/Output):** Περιέχει τις συναρτήσεις για είσοδο και έξοδο υψηλότερου επιπέδου. Χρησιμοποιείται για τη συνάρτηση perror και για τη συνάρτηση sprintf().

Μέρος 2.1

- Ζητούμενο:

Να επεκταθεί το πρόγραμμα του 1^{ου} μέρους, ώστε:

1. Να δημιουργηθεί μία διεργασία παιδί που θα "χαιρετάει τον κόσμο" αναφέροντας το αναγνωριστικό της και το αναγνωριστικό του γονέα της.
2. Ο γονέας να τυπώνει το αναγνωριστικό του παιδιού και να περιμένει τον τερματισμό του.
3. Να οριστεί μια μεταβλητή x, στο γονέα πριν τη δημιουργία του παιδιού, και να ανατεθεί διαφορετική τιμή στο γονέα και στο παιδί και να τυπωθεί η τιμή της.
4. Να ανατεθεί στη διεργασία παιδί, η αναζήτηση του χαρακτήρα στο αρχείο.
5. Να δημιουργηθεί μία διεργασία παιδί που να εκτελεί τον κώδικα που δόθηκε στο Ερώτημα 1.

- Κώδικας 2.1

```

1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 #include <stdio.h>
5 #include <sys/wait.h>
6
7 int main(int argc, char *argv[]) {
8     if (argc != 2) {
9         fprintf(stderr, "Usage: program input output char\n", stderr);
10        exit(1);
11    }
12    int fd_in = open(argv[1], O_RDONLY);
13    if (fd_in < 0) {
14        perror("open input");
15        exit(1);
16    }
17
18    int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
19    if (fd_out < 0) {
20        perror("open output");
21        exit(1);
22    }
23
24    int x = 10;
25    printf("[Parent] Variable x initialized to: %d\n", x);
26    pid_t pid = fork();
27    if (pid < 0) {
28        perror("fork failed");
29        exit(1);
30    }
31    else if (pid == 0) {
32        printf("[Child] Hello World! My PID is %d. My parent's PID is %d.\n", getpid(), getppid());
33        x = 20;
34        printf("[Child] I changed x to: %d\n", x);
35        char buffer[1024];
36        ssize_t bytes;
37        char target = argv[3][0];
38        int count = 0;
39        while ((bytes = read(fd_in, buffer, sizeof(buffer))) > 0) {
40            for (int i = 0; i < bytes; i++) {
41                if (buffer[i] == target) {
42                    count++;
43                }
44            }
45        }
46        if (bytes < 0) {
47            perror("read");
48            exit(1);
49        }
50        char result[50];
51        int len = sprintf(result, "Count: %d\n", count);
52        write(fd_out, result, len);
53        exit(0);
54    }
55    else {
56        printf("[Parent] I created the child whose PID is: %d\n", pid);
57        x = 30;
58        printf("[Parent] I changed x to: %d\n", x);
59        wait(NULL);
60        printf("[Parent] Child, terminated.\n");
61        close(fd_in);
62        close(fd_out);
63    }
64    return 0;
65 }

```

- Επεξήγηση κώδικα

Ο κώδικας είναι εξελιγμένη έκδοση του προηγούμενου. Η βασική λογική του, δηλαδή η μέτρηση χαρακτήρα παραμένει, αλλά τώρα εκτελείται από ένα child process που δημιουργείται με fork().

- Πετυχαίνει τον στόχο του;

ΝΑΙ!

- Επεξήγηση τμημάτων του κώδικα

Άνοιγμα Αρχείων

```
int fd_in = open(argv[1], O_RDONLY);
int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

Τα αρχεία ανοίγουν από τον parent πριν το fork(). Αυτό σημαίνει ότι και τα δύο processes μοιράζονται τους ίδιους file descriptors μετά το fork.

Η Μεταβλητή x και η Έννοια του Virtual Memory

```
int x = 10;
printf("[Parent] Variable x initialized to: %d\n", x);
pid_t pid = fork();
```

Η μεταβλητή x αρχικοποιείται πριν το fork(), οπότε και τα δύο processes ξεκινούν με x = 10. Στη συνέχεια, παρόλο που φαίνεται ότι μοιράζονται την ίδια μεταβλητή, δεν συμβαίνει αυτό. Το λειτουργικό χρησιμοποιεί Copy-on-Write (CoW) και η μνήμη αντιγράφεται μόνο όταν κάποιο process τη τροποποιήσει. Έτσι κάθε process έχει το δικό του αντίγραφο του x.

Το fork() και οι Τρεις Περιπτώσεις

```
pid_t pid = fork();
if (pid < 0) { /* Αποτυχία */ }
else if (pid == 0) { /* Child process */ }
else { /* Parent process */ }
```

Γιατί το wait(NULL) Είναι Απαραίτητο

Αν ο parent δεν περίμενε το child, τότε το child θα γινόταν Zombie process. Δηλαδή τερματισμένο αλλά η καταχώρησή του στον process table παραμένει μέχρι ο parent να διαβάσει τον κωδικό εξόδου. Χρησιμοποιώντας το wait(NULL), ο parent αγνοεί τον κωδικό εξόδου αλλά αποτρέπει το zombie.

- System Calls που χρησιμοποιεί ο κώδικας
 - open()
 - read()
 - write()
 - close()
 - exit()
 - fork()
 - getpid()
 - getppid()
 - wait(NULL)

Γιατί χρειάζονται οι 5 βιβλιοθήκες;

Κάθε header file δίνει στο πρόγραμμά πρόσβαση σε συγκεκριμένες συναρτήσεις ή σταθερές που χρησιμοποιούνται παρακάτω στον κώδικα:

1. **<fcntl.h> (File Control):** Περιέχει τα flags και τις σταθερές που χρειάζονται για το άνοιγμα των αρχείων. Στον κώδικά, παρέχει τις σταθερές O_RDONLY, O_WRONLY, O_CREAT και O_TRUNC, οι οποίες περνιούνται ως ορίσματα στη συνάρτηση open().
2. **<unistd.h> (Unix Standard):** Παρέχει πρόσβαση στο API του λειτουργικού συστήματος (POSIX) για βασικές λειτουργίες I/O χαμηλού

επιπέδου. Παρέχει τις βασικές συναρτήσεις διαχείρισης αρχείων: `read()`, `write()` και `close()`. Επίσης, ορίζει τον τύπο `ssize_t`.

3. **<stdlib.h> (Standard Library):** Είναι η γενική βιβλιοθήκη της C για βασικές λειτουργίες όπως η διαχείριση μνήμης, μετατροπές τύπων και ο έλεγχος του προγράμματος. Στον κώδικά, χρησιμοποιείται αποκλειστικά για τη συνάρτηση `exit(1)`, η οποία τερματίζει απότομα το πρόγραμμα επιστρέφοντας έναν κωδικό σφάλματος στο λειτουργικό σύστημα αν κάτι πάει στραβά.
4. **<stdio.h> (Standard Input/Output):** Περιέχει τις συναρτήσεις για είσοδο και έξοδο υψηλότερου επιπέδου. Χρησιμοποιείται για τη συνάρτηση `perror` και για τη συνάρτηση `sprintf()`.
5. **<sys/wait.h>:** χρησιμοποιείται για να συγχρονίσει μια parent process με τις child processes που η ίδια δημιουργεί μέσω της εντολής `fork()`.

■ Κώδικας 2.2

```
1  #include <unistd.h>
2  #include <stdlib.h>
3  #include <stdio.h>
4  #include <sys/wait.h>
5
6  int main(int argc, char *argv[]) {
7      if (argc != 4) {
8          printf("Usage: %s input output char\n", argv[0]);
9          exit(1);
10     }
11     pid_t pid = fork();
12     if (pid < 0) {
13         perror("fork failed");
14         exit(1);
15     }
16     else if (pid == 0) {
17         char *args[] = {"/part1", argv[1], argv[2], argv[3], NULL};
18         execv(path: "/part1", args);
19         perror("execv failed");
20         exit(1);
21     }
22     else {
23         wait(NULL);
24         printf("[Parent] The child (through syscall: execv) has completed the execution.\n");
25     }
26     return 0;
27 }
```

■ Επεξήγηση κώδικα

Ο κώδικας αυτός είναι ένα launcher. Ο parent δημιουργεί ένα child process το οποίο αντικαθιστά τον εαυτό του με ένα άλλο πρόγραμμα (./part1) χρησιμοποιώντας το system call `execv()`.

- Πετυχαίνει τον στόχο του;

ΝΑΙ!

- Επεξήγηση τμημάτων του κώδικα

Έλεγχος Ορισμάτων

```
if (argc != 4) {  
    printf("Usage: %s input output char\n", argv[0]);  
    exit(1);  
}
```

Ίδιος έλεγχος με πριν. Απαιτούνται 3 ορίσματα. Εδώ χρησιμοποιείται `printf` αντί για `write()` στο `stderr`.

fork() - Δημιουργία Child

```
pid_t pid = fork();
```

Δημιουργείται ένα αντίγραφο της τρέχουσας διεργασίας. Από αυτό το σημείο εκτελούνται δύο ανεξάρτητες διεργασίες.

Child Process - *execv()*

```
char *args[] = {"/part1", argv[1], argv[2], argv[3], NULL};  
execv("/part1", args);  
perror("execv failed");  
exit(1);
```

Το `execv()` αντικαθιστά πλήρως το τρέχον process image με ένα νέο πρόγραμμα. Αυτό σημαίνει πως ο κώδικας, η στοίβα και το heap, αντικαθίστανται. Το PID παραμένει το ίδιο και δεν δημιουργείται νέα διεργασία. Τέλος, δεν επιστρέφει ποτέ αν πετύχει. Αν φτάσει στη `perror()`, σημαίνει ότι απέτυχε.

Array() - args[]

```
char *args[] = {"/part1", argv[1], argv[2], argv[3], NULL};  
//      ↑ argv[0] ↑ input ↑ output ↑ char ↑ τερματισμός
```

Το NULL στο τέλος είναι απαραίτητο έτσι ώστε το `execv()` να ξέρει πού τελειώνουν τα ορίσματα.

- System Calls που χρησιμοποιεί ο κώδικας
 - `execv()`
 - `wait(NULL)`

Γιατί χρειάζονται οι 4 βιβλιοθήκες;

Κάθε header file δίνει στο πρόγραμμά πρόσβαση σε συγκεκριμένες συναρτήσεις ή σταθερές που χρησιμοποιούνται παρακάτω στον κώδικα:

1. **<unistd.h> (Unix Standard):** Παρέχει πρόσβαση στο API του λειτουργικού συστήματος (POSIX) για βασικές λειτουργίες I/O χαμηλού επιπέδου. Παρέχει τις βασικές συναρτήσεις διαχείρισης αρχείων: read(), write() και close(). Επίσης, ορίζει τον τύπο ssize_t.
2. **<stdlib.h> (Standard Library):** Είναι η γενική βιβλιοθήκη της C για βασικές λειτουργίες όπως η διαχείριση μνήμης, μετατροπές τύπων και ο έλεγχος του προγράμματος. Στον κώδικά, χρησιμοποιείται αποκλειστικά για τη συνάρτηση exit(1), η οποία τερματίζει απότομα το πρόγραμμα επιστρέφοντας έναν κωδικό σφάλματος στο λειτουργικό σύστημα αν κάτι πάει στραβά.
3. **<stdio.h> (Standard Input/Output):** Περιέχει τις συναρτήσεις για είσοδο και έξοδο υψηλότερου επιπέδου. Χρησιμοποιείται για τη συνάρτηση perror και για τη συνάρτηση sprintf().
4. **<sys/wait.h>:** χρησιμοποιείται για να συγχρονίσει μια parent process με τις child processes που η ίδια δημιουργεί μέσω της εντολής fork().

Μέρος 3

▪ Ζητούμενο:

Να επεκταθεί το πρόγραμμα του Μέρους 2, ώστε να δημιουργεί P διεργασίες παιδιά οι οποίες θα αναζητούν παράλληλα το χαρακτήρα στο αρχείο, ενώ η γονεϊκή διεργασία θα συλλέγει και θα τυπώνει το συνολικό αποτέλεσμα. Όταν το πρόγραμμά σας θα δέχεται Control+C από το πληκτρολόγιο (SIGINT signal) θα πρέπει αντί να τερματίζει, να τυπώνει το συνολικό αριθμό διεργασιών που αναζητούν το αρχείο.

■ Κώδικας

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #include <fcntl.h>
6 #include <signal.h>
7
8 #define P 3
9
10 volatile sig_atomic_t active_children = 0;
11
12 void safe_print_int(int num) {
13     char buf[32];
14     int i = 0;
15     if (num == 0) {
16         write(STDOUT_FILENO, buf, "0", nbytes: 1);
17         return;
18     }
19     while (num > 0) {
20         buf[i++] = (num % 10) + '0';
21         num /= 10;
22     }
23     for (int j = i - 1; j >= 0; j--) {
24         write(STDOUT_FILENO, &buf[j], nbytes: 1);
25     }
26 }
27
28 void sigint_handler(int sig) {
29     write(STDOUT_FILENO, buf, "\n[SIGNAL] Control+C caught! Active search processes: ", nbytes: 53);
30     safe_print_int(active_children);
31     write(STDOUT_FILENO, buf, "\n", nbytes: 1);
32 }
33
34 int main(int argc, char *argv[]) {
35     if (argc != 4) {
36         write(STDERR_FILENO, buf, "Usage: ./askisi3 input_file output_file char\n", nbytes: 45);
37         exit(1);
38     }
39
40     struct sigaction sa;
41     sa.sa_handler = sigint_handler;
42     sigemptyset(&sa.sa_mask);
43     sa.sa_flags = SA_RESTART;
44     sigaction(SIGINT, &sa, NULL);
45
46     int pipefd[2];
47     if (pipe(pipefd) == -1) {
48         perror("pipe failed");
49         exit(1);
50     }
51
52     int fd_temp = open(argv[1], O_RDONLY);
53     if (fd_temp < 0) {
54         perror("open input");
55         exit(1);
56     }
57     off_t file_size = lseek(fd_temp, 0, SEEK_END);
58     close(fd_temp);
59
60     off_t chunk_size = file_size / P;
61
62     printf("[PARENT] Starting creation of %d children...\n", P);
63     printf("[PARENT] File size is %ld bytes. You have 5 seconds to press Ctrl+C!\n", (long)file_size);
64
65     for (int i = 0; i < P; i++) {
66         pid_t pid = fork();
67
68         if (pid < 0) {
69             perror("fork failed");
70             exit(1);
71         }
72         else if (pid == 0) {
73             signal(SIGINT, SIG_IGN);
74             close(pipefd[0]);
75
76             sleep(5);
77
78             int fd_in = open(argv[1], O_RDONLY);
79             off_t start = i * chunk_size;
80             off_t end = (i == P - 1) ? file_size : start + chunk_size;
81
82             lseek(fd_in, start, SEEK_SET);
83
84             char buffer[1024];
85             ssize_t bytes;
86             char target = argv[3][0];
87             int local_count = 0;
88             off_t bytes_to_read = end - start;
89             off_t total_read = 0;
90
91             while (total_read < bytes_to_read) {
92                 size_t to_read = sizeof(buffer);
93                 if (bytes_to_read - total_read < sizeof(buffer)) {
94                     to_read = bytes_to_read - total_read;
95                 }
96
97                 bytes = read(fd_in, buffer, to_read);
98                 if (bytes <= 0) break;
99             }
100         }
101     }
102 }
```

```

181         for (int j = 0; j < bytes; j++) {
182             if (buffer[j] == target) local_count++;
183         }
184         total_read += bytes;
185     }
186     close(fd_in);
187
188     write(pipefd[1], &local_count, sizeof(int));
189     close(pipefd[1]);
190     exit(0);
191 }
192 else {
193     active_children++;
194 }
195 }
196 close(pipefd[1]);
197
198 int total_count = 0;
199 int partial_count;
200
201 for (int i = 0; i < P; i++) {
202     if (read(pipefd[0], &partial_count, sizeof(int)) > 0) {
203         total_count += partial_count;
204     }
205 }
206 close(pipefd[0]);
207
208 pid_t pid;
209 int status;
210 while ((pid = wait(&status)) > 0) {
211     active_children--;
212 }
213
214 int fd_out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
215 if (fd_out >= 0) {
216     char result_msg[50];
217     int len = sprintf(result_msg, "Total Count: %d\n", total_count);
218     write(fd_out, result_msg, len);
219     close(fd_out);
220 }
221
222 printf("[PARENT] Search completed. Found %d occurrences.\n", total_count);
223
224 return 0;
225

```

- Επεξήγηση κώδικα

Ο κώδικας υλοποιεί παράλληλη μέτρηση χαρακτήρα σε αρχείο, χωρίζοντάς το σε P (P = 3) τμήματα που επεξεργάζονται ταυτόχρονα από 3 child processes. Επιπλέον, χειρίζεται το σήμα SIGINT (Ctrl+C).

- Πετυχαίνει τον στόχο του;

ΝΑΙ!

- Επεξήγηση τμημάτων του κώδικα

Καθολικές Μεταβλητές & volatile sig_atomic_t

```

#define P 3
volatile sig_atomic_t active_children = 0;

```

P: Ο αριθμός των child processes, άρα των τμημάτων του αρχείου
 volatile sig_atomic_t: ο ειδικός τύπος για τις μεταβλητές που τροποποιούνται μέσα σε signal handlers. Το volatile εμποδίζει τον compiler να την κάνει cache σε register, και το sig_atomic_t εγγυάται atomic (αδιαίρετη) ανάγνωση/εγγραφή.

safe_print_int() — Async-Signal-Safe Εκτύπωση

```

void safe_print_int(int num) {
    char buf[32];
    int i = 0;

```

```

while (num > 0) {
    buf[i++] = (num % 10) + '0';
    num /= 10;
}
for (int j = i - 1; j >= 0; j--)
    write(STDOUT_FILENO, &buf[j], 1);
}

```

Εκτυπώνει έναν ακέραιο χρησιμοποιώντας μόνο write() και όχι printf(). Η printf() δεν είναι async-signal-safe, και έτσι, αν διακοπεί από signal ενώ χρησιμοποιεί εσωτερικά locks για το buffer της, μπορεί να προκαλέσει deadlock. Μέσα σε signal handler επιτρέπονται μόνο async-signal-safe συναρτήσεις, όπως η write().

Signal Handler για SIGINT

```

void sigint_handler(int sig) {
    write(STDOUT_FILENO, "\n[SIGNAL] Control+C caught! Active search processes: ", 53);
    safe_print_int(active_children);
    write(STDOUT_FILENO, "\n", 1);
}

```

Όταν ο χρήστης πατήσει Ctrl+C, Ο parent δεν τερματίζει, απλώς εκτυπώνει πόσα children εκτελούνται ακόμα. Τα children αγνοούν το SIGINT.

Εγκατάσταση Signal Handler με sigaction()

```

struct sigaction sa;
sa.sa_handler = sigint_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART;
sigaction(SIGINT, &sa, NULL);

```

Πεδίο	Τιμή	Σημασία
sa_handler	sigint_handler	Η συνάρτηση που θα κληθεί
sa_mask	κενή	Κανένα επιπλέον signal δεν μπλοκάρεται κατά τη διάρκεια του handler
sa_flags = SA_RESTART		Οι system calls που διακόπηκαν από το signal επανεκκινούν αυτόματα αντί να επιστρέψουν σφάλμα

Η sigaction() είναι προτιμότερη από την signal() γιατί έχει πιο προβλέψιμη συμπεριφορά across platforms.

Δημιουργία Pipe

```

int pipefd[2];
pipe(pipefd);
// pipefd[0] = read end
// pipefd[1] = write end

```

Το pipe είναι ο μηχανισμός επικοινωνίας (IPC) μεταξύ children και parent. Κάθε child γράφει τον τοπικό του μετρητή στο pipe, και ο parent τους διαβάζει και τους αθροίζει.

Υπολογισμός Μεγέθους Αρχείου με lseek()

```
int fd_temp = open(argv[1], O_RDONLY);
off_t file_size = lseek(fd_temp, 0, SEEK_END);
close(fd_temp);
off_t chunk_size = file_size / P;
```

Παράμετρος lseek	Τιμή	Σημασία
fd	fd_temp	Το αρχείο
offset	0	Μετακίνηση κατά 0 bytes
whence	SEEK_END	Από το τέλος του αρχείου

Επιστρέφει τη θέση μετά την κίνηση, δηλαδή το συνολικό μέγεθος του αρχείου σε bytes. Στη συνέχεια διαιρείται σε P ίσα chunks.

Δημιουργία Children & Παράλληλη Επεξεργασία

```
for (int i = 0; i < P; i++) {
    pid_t pid = fork();
    if (pid == 0) {
        signal(SIGINT, SIG_IGN); // Child αγνοεί Ctrl+C
        close(pipefd[0]);        // Child δεν χρειάζεται read end
        sleep(5);                // Παράθυρο για Ctrl+C demo
        ...
    } else {
        active_children++;
    }
}
```

Κατανομή Τμημάτων

```
off_t start = i * chunk_size;
off_t end = (i == P - 1) ? file_size : start + chunk_size;
lseek(fd_in, start, SEEK_SET);
```

Ανάγνωση Chunk

```
off_t bytes_to_read = end - start;
off_t total_read = 0;
```

```
while (total_read < bytes_to_read) {
    size_t to_read = sizeof(buffer);
    if (bytes_to_read - total_read < sizeof(buffer))
        to_read = bytes_to_read - total_read;
```

```
    bytes = read(fd_in, buffer, to_read);
    ...
    total_read += bytes;
}
```


Parent - Συλλογή Αποτελεσμάτων

```
close(pipefd[1]); // O parent κλείνει το write end — ΚΡΙΣΙΜΟ
int total_count = 0;
int partial_count;
for (int i = 0; i < P; i++) {
    if (read(pipefd[0], &partial_count, sizeof(int)) > 0)
        total_count += partial_count;
}
close(pipefd[0]);
```

Ο parent πρέπει να κλείσει το pipefd[1]. Αν δεν το κάνει, το pipe δεν θα φτάσει ποτέ σε EOF και η read() θα μπλοκάρει για πάντα. Στη συνέχεια ο parent αθροίζει τα P μερικά αποτελέσματα.

wait() & Καθαρισμός

```
while ((wpid = wait(&status)) > 0)
    active_children--;
```

Ο parent περιμένει **όλα** τα children να τερματίσουν (όχι μόνο ένα), αποτρέποντας zombie processes, και ενημερώνει τον active_children counter.

- System Calls που χρησιμοποιεί ο κώδικας
 - pipe()
 - lseek()
 - sigaction()
 - signal()
 - sleep()

Γιατί χρειάζονται οι 6 βιβλιοθήκες;

Κάθε header file δίνει στο πρόγραμμά πρόσβαση σε συγκεκριμένες συναρτήσεις ή σταθερές που χρησιμοποιούνται παρακάτω στον κώδικα:

1. **<fcntl.h> (File Control):** Περιέχει τα flags και τις σταθερές που χρειάζονται για το άνοιγμα των αρχείων. Στον κώδικά, παρέχει τις σταθερές O_RDONLY, O_WRONLY, O_CREAT και O_TRUNC, οι οποίες περνιούνται ως ορίσματα στη συνάρτηση open().
2. **<unistd.h> (Unix Standard):** Παρέχει πρόσβαση στο API του λειτουργικού συστήματος (POSIX) για βασικές λειτουργίες I/O χαμηλού επιπέδου. Παρέχει τις βασικές συναρτήσεις διαχείρισης αρχείων: read(), write() και close(). Επίσης, ορίζει τον τύπο ssize_t.
3. **<stdlib.h> (Standard Library):** Είναι η γενική βιβλιοθήκη της C για βασικές λειτουργίες όπως η διαχείριση μνήμης, μετατροπές τύπων και ο έλεγχος του προγράμματος. Στον κώδικά, χρησιμοποιείται αποκλειστικά για τη συνάρτηση exit(1), η οποία τερματίζει απότομα το πρόγραμμα

επιστρέφοντας έναν κωδικό σφάλματος στο λειτουργικό σύστημα αν κάτι πάει στραβά.

4. **<stdio.h> (Standard Input/Output):** Περιέχει τις συναρτήσεις για είσοδο και έξοδο υψηλότερου επιπέδου. Χρησιμοποιείται για τη συνάρτηση perror και για τη συνάρτηση sprintf().
5. **<sys/wait.h>:** χρησιμοποιείται για να συγχρονίσει μια parent process με τις child processes που η ίδια δημιουργεί μέσω της εντολής fork().
6. **<signal.h>:** χρησιμοποιείται για να συγχρονίσει μια parent process με τις child processes που η ίδια δημιουργεί μέσω της εντολής fork().

Μέρος 4

▪ Ζητούμενο:

Το ζητούμενο είναι η ανάπτυξη μιας παράλληλης εφαρμογής καταμέτρησης χαρακτήρων σε αρχείο, η οποία οργανώνεται σε τρία ανεξάρτητα επίπεδα διεργασιών: το Front-end για τη λήψη εντολών από τον χρήστη, τον Dispatcher για τον συντονισμό και την κυκλική κατανομή του φόρτου, και τους δυναμικά μεταβαλλόμενους Workers που αναλαμβάνουν το πρακτικό κομμάτι της μέτρησης. Η επικοινωνία μεταξύ των διεργασιών πρέπει να γίνεται αποκλειστικά μέσω Pipes (με χρήση σημάτων για την αναφορά προόδου), ενώ η εφαρμογή απαιτείται να είναι ανθεκτική σε σφάλματα, διασφαλίζοντας ότι αν ένας Worker τερματιστεί απροσδόκητα, ο Dispatcher θα του πάρει την ημιτελή εργασία και θα την αναθέσει σε άλλον εργάτη ώστε να ολοκληρωθεί ομαλά η αναζήτηση.

▪ Υλοποίηση Μέρους 3

Η υλοποίηση του μέρους 3 της άσκησης περιλαμβάνει δύο ξεχωριστά αρχεία κώδικα C. Το αρχείο `main.c` και το αρχείο `worker.c`

- Αρχείο: `main.c`

Ο κώδικας του αρχείου `main.c` αποτελεί την πλήρη υλοποίηση σε γλώσσα C του συστήματος Front-end και Dispatcher για την παράλληλη εφαρμογή καταμέτρησης χαρακτήρων που περιγράφηκε στο προηγούμενο κείμενο.

Ουσιαστικά, συντονίζει την επικοινωνία με τον χρήστη και τη διαχείριση των εργατών (workers). Ας δούμε αναλυτικά τι κάνει κάθε βασικό τμήμα του:

1. Η συνάρτηση `main` (Αρχειοποίηση)

- Ξεκινάει ζητώντας το όνομα του αρχείου ως όρισμα από τη γραμμή εντολών.
- Δημιουργεί δύο ανώνυμα Pipes για την αμφίδρομη επικοινωνία μεταξύ Front-end και Dispatcher.
- Κάνει `fork()`: η θυγατρική διεργασία τρέχει τη συνάρτηση `run_dispatcher` και η γονική τη `run_frontend`.

2. Το Front-End (`run_frontend`)

Αυτή η διεργασία αλληλεπιδρά με τον χρήστη.

- Χρησιμοποιεί την κλήση συστήματος `select()`, η οποία της επιτρέπει να περιμένει ταυτόχρονα δεδομένα από δύο πηγές: το πληκτρολόγιο (εντολές χρήστη) και το pipe του Dispatcher (απαντήσεις/αποτελέσματα), χωρίς να "κολλάει" (block) σε κανένα από τα δύο.
- Διαχείριση εντολών:
 - `add, search <char>, exit`: Τις μετατρέπει σε κατάλληλα μηνύματα (π.χ. "ADD", "SEARCH c") και τις στέλνει μέσω pipe στον Dispatcher.
 - `status`: Δεν στέλνει μήνυμα μέσω pipe. Αντίθετα, στέλνει το σήμα `SIGUSR1` στον Dispatcher, ικανοποιώντας την απαίτηση της άσκησης για διακοπή μέσω σήματος.

3. Ο Dispatcher (`run_dispatcher`)

Αυτή είναι η "καρδιά" του συστήματος. Εκτελείται σε έναν ατέρμονα βρόχο (`while(1)`) ελέγχοντας συνεχώς, με μη-μπλοκάροντα (non-blocking) τρόπο, διάφορα γεγονότα:

- Ανθεκτικότητα (Fault Tolerance): Χρησιμοποιεί την `waitpid(-1, ..., WNOHANG)` για να δει αν κάποιος worker "πέθανε" (π.χ. τερματίστηκε βίαια). Αν εντοπίσει νεκρό worker, μαρκάρει το κομμάτι του αρχείου (chunk) που του είχε αναθέσει ξανά ως "μη ανατεθειμένο" (`status = 0`), ώστε να δοθεί σε άλλον εργάτη.
- Αναφορά Προόδου: Αν δεχτεί το σήμα `SIGUSR1`, ενεργοποιείται μια μεταβλητή (flag). Ο Dispatcher το βλέπει, υπολογίζει το ποσοστό (% ολοκλήρωσης = `completed chunks / total chunks`) και στέλνει την αναφορά στο Front-end.
- Διαχείριση Εντολών:
 - Αν λάβει `ADD`: Κάνει `fork()` και στη συνέχεια `execv("./worker", ...)` για να μετατρέψει τη νέα διεργασία στο εκτελέσιμο του εργάτη. Της περνάει ως ορίσματα τα file descriptors των pipes για να ξέρει πού να γράφει.
 - Αν λάβει `SEARCH`: Διαβάζει το μέγεθος του αρχείου με τη `stat()`, το "κόβει" σε λογικά chunks των 1024 bytes και αρχικοποιεί τον πίνακα chunks.
- Κατανομή Εργασίας: Διατρέχει τον πίνακα με τα chunks. Αν βρει ελεύθερο chunk και ελεύθερο worker, του στέλνει μήνυμα `TSK <offset> <size> <char>`.
- Συλλογή Αποτελεσμάτων: Διαβάζει απαντήσεις τύπου `RES <worker_id> <βρέθηκαν>` από τους workers, αθροίζει τους χαρακτήρες, ελευθερώνει τον worker και μαρκάρει το chunk ως ολοκληρωμένο.
- Αρχείο: `worker.c`

Ο συγκεκριμένος κώδικας προορίζεται να μεταγλωττιστεί σε ένα αυτόνομο εκτελέσιμο (π.χ., με `gcc worker.c -o worker`), το οποίο καλείται και ελέγχεται δυναμικά από τον Dispatcher μέσω της εντολής `execv()`.

Ας δούμε αναλυτικά τι κάνει:

Η Λειτουργία του Worker

1. Αρχικοποίηση μέσω Ορισμάτων:

Όταν ο Dispatcher "γεννάει" (`fork`) αυτόν τον worker, του περνάει 4 ορίσματα:

- Το ID του εργάτη.
- Το File Descriptor Εισόδου (`fd_in`): Το άκρο του pipe από όπου θα διαβάζει εντολές.
- Το File Descriptor Εξόδου (`fd_out`): Το άκρο του pipe όπου θα στέλνει τα αποτελέσματα.

- ο Το Όνομα του αρχείου που θα αναλυθεί (το οποίο ανοίγει αμέσως για ανάγνωση).

2. Ατέρμονος Βρόχος Εργασίας (Event Loop):

Ο Worker δεν τερματίζει από μόνος του. Μπαίνει σε μια λούπα `while(read(...))` περιμένοντας να λάβει δεδομένα από τον Dispatcher. Αν το pipe κλείσει (π.χ. αν ο Dispatcher στείλει SIGKILL με την εντολή `exit`), η `read` αποτυγχάνει και ο Worker τερματίζει.

3. Εκτέλεση του Chunk (Αναζήτηση):

Μόλις λάβει ένα μήνυμα εργασίας (μορφής TSK <offset> <size> <char>):

- ο Χρησιμοποιεί την κλήση `lseek(file_fd, offset, SEEK_SET)` για να μετακινήσει τον "κέρσορα" του αρχείου ακριβώς στο σημείο (σε bytes) που του ζητήθηκε.
- ο Ξεκινάει να διαβάζει το κομμάτι του αρχείου σε δικά του, μικρότερα τοπικά chunks των 1024 bytes (για αποδοτικότητα μνήμης), μέχρι να συμπληρώσει το συνολικό size που του ανατέθηκε.
- ο Καταμετρά τις εμφανίσεις του χαρακτήρα-στόχου.

4. Προσομοίωση Φόρτου (usleep):

Η εντολή `usleep(500000)` "κοιμίζει" τον Worker για μισό δευτερόλεπτο (500 ms) μετά από κάθε chunk. Αυτή είναι μια πολύ έξυπνη προσθήκη: επειδή οι σύγχρονοι υπολογιστές είναι πολύ γρήγοροι, η αναζήτηση θα τελείωνε ακαριαία. Η καθυστέρηση αυτή βάζει ένα "τεχνητό φρένο" ώστε να προλαβαίνεις να χρησιμοποιήσεις την εντολή `status` στο Front-end και να δεις την παράλληλη εκτέλεση στην πράξη!

5. Επιστροφή Αποτελέσματος:

Μόλις τελειώσει τον υπολογισμό, στέλνει ένα τυποποιημένο μήνυμα των 16 χαρακτήρων (π.χ. RES 00002 00045, δηλαδή "Ο Worker 2 βρήκε 45 εμφανίσεις") πίσω στον Dispatcher μέσω του `fd_out`.

Σε συνδυασμό με το προηγούμενο αρχείο, η εφαρμογή σου ικανοποιεί πλήρως όλες τις απαιτήσεις του κειμένου.

- Πετυχαίνει τον στόχο του;

ΝΑΙ!

Γιατί χρειάζονται οι 6 βιβλιοθήκες;

1. <string.h> (Διαχείριση Strings και Μνήμης)

Η C δεν έχει ενσωματωμένο τύπο δεδομένων για κείμενο, αλλά χρησιμοποιεί πίνακες χαρακτήρων που τερματίζουν με τον χαρακτήρα \0.

Η <string.h> παρέχει τα εργαλεία για απευθείας διαχείριση μνήμης και strings.

- **Χρήση για Strings:**

- strlen(s): μήκος string
- strcpy(dest, src) / strncpy(): Αντιγράφει ένα string μέσα σε ένα άλλο
- strcmp(s1, s2): συγκρίνει δύο strings αλφαριθμητικά
- strcat(dest, src): ενώνει ένα string στο τέλος ενός άλλου

- **Χρήση για Μνήμη (Raw Memory):**

- memset(ptr, value, num): γεμίζει ένα μπλοκ μνήμης με μια συγκεκριμένη τιμή
- memcpy(dest, src, num): αντιγράφει num bytes μνήμης από ένα σημείο σε ένα άλλο

2. <signal.h> (Διαχείριση Σημάτων - Signals)

Τα σήματα είναι ένας τρόπος με τον οποίο το λειτουργικό σύστημα στέλνει ειδοποιήσεις (ασύγχρονα γεγονότα) στο πρόγραμμά, διακόπτοντας προσωρινά την κανονική του ροή. Η βιβλιοθήκη αυτή σου επιτρέπει να πιάσεις, να αγνοήσεις ή να στείλεις τέτοια σήματα.

- **Συνήθη Σήματα:**

- SIGINT: αποστέλλεται όταν ο χρήστης πατήσει Ctrl+C στο τερματικό.
- SIGTERM: αίτημα τερματισμού (kill)
- SIGALRM: σήμα-ξυπνητήρι από έναν timer

- **Βασικές Συναρτήσεις:**

- sigaction(): επιτρέπει να ορίσεις μια δική σου συνάρτηση handler που θα εκτελεστεί αυτόματα όταν ληφθεί ένα συγκεκριμένο σήμα. Για παράδειγμα, αντί το πρόγραμμά να κλείσει απότομα με Ctrl+C, μπορείς να το "πιάσεις", να σώσεις τα δεδομένα του χρήστη, να κλείσεις τα ανοιχτά αρχεία και μετά να τερματίσεις ομαλά.
- kill(pid, signal): Στέλνει ένα σήμα σε μια άλλη διεργασία (βάσει του PID της).

3. <sys/stat.h> (Πληροφορίες και Μεταδεδομένα Αρχείων)

Ενώ η `<fcntl.h>` χρησιμοποιείται για να ανοίξεις αρχεία, η `<sys/stat.h>` χρησιμοποιείται για να μάθεις πληροφορίες γι' αυτά χωρίς απαραίτητα να τα ανοίξεις (metadata).

- **Βασική Συνάρτηση:** Η `stat()` (ή `fstat()` αν έχεις ήδη ανοιχτό το αρχείο). Δίνει σε αυτήν τη συνάρτηση τη διαδρομή ενός αρχείου και ένα "άδειο" struct τύπου `stat`, και το λειτουργικό σύστημα το γεμίζει με δεδομένα.
- **Τι μπορείς να μάθεις (Χρήση):**
 - Το μέγεθος του αρχείου σε bytes (`st_size`).
 - Ποιος είναι ο ιδιοκτήτης του αρχείου (`st_uid`).
 - Τα δικαιώματα (permissions) του αρχείου (`st_mode`).
 - Πότε τροποποιήθηκε τελευταία φορά (`st_mtime`).
- **Χρήσιμες Μακροεντολές:** Διαθέτει εργαλεία για να ρωτήσεις τι είδους αρχείο είναι. Π.χ. `S_ISDIR(mode)` (Είναι φάκελος;), `S_ISREG(mode)` (Είναι κανονικό αρχείο κειμένου/δεδομένων;).

4. `<sys/select.h>` (Πολυπλεξία Εισόδου/Εξόδου - I/O Multiplexing)

Αυτή η βιβλιοθήκη λύνει το πρόβλημα του "πώς ακούω πολλά πράγματα ταυτόχρονα χωρίς να παγώσει το πρόγραμμά μου". Είναι κρίσιμη σε προγράμματα δικτύου (servers) ή όταν διαβάζεις από πολλά αρχεία/pipes.

- **Βασικά Εργαλεία:** Χρησιμοποιεί μακροεντολές για να φτιάξεις αυτή τη λίστα αναμονής, όπως:
 - `FD_ZERO()`: Αδειάζει τη λίστα.
 - `FD_SET()`: Προσθέτει ένα αρχείο/socket στη λίστα.
 - `FD_ISSET()`: Μετά το ξύπνημα της `select()`, ελέγχει ποιο αρχείο ήταν τελικά αυτό που είχε δεδομένα.